

Capstone Project

Automated Application Delivery on AWS Using Jenkins, Terraform, Ansible, and ECS

Introduction

Modern application delivery involves application validation, infrastructure provisioning, configuration management, security scanning, deployment, and monitoring. Automating these activities creates a repeatable workflow and reduces manual execution.

In this project, a React + Vite Quiz App is containerized with Docker and deployed to Amazon ECS. Terraform provisions the infrastructure, Jenkins automates infrastructure and application delivery, and Ansible configures a dedicated managed EC2 instance hosting DevOps services. SonarQube and Trivy provide security and code analysis, ECR stores container images, and ECS uses Blue/Green deployment behind an Application Load Balancer. CloudWatch provides monitoring data, while Grafana uses it for dashboards and alerting.

Technology Stack

Terraform	Infrastructure Provisioning
Ansible	Server Configuration
Git, AWS CodeCommit	Source Code Management
Jenkins	Pipeline Automation
Amazon EC2	DevOps Server
Docker	Application Containerization
Amazon ECR	Image Storage
Amazon ECS	Container Deployment
Application Load Balancer	Traffic Management
SonarQube	Code Quality and Security Analysis
Trivy	Container Vulnerability Scanning

Amazon CloudWatch	Logs and Metrics
Grafana	Monitoring Dashboards
Amazon VPC	Network Infrastructure
Internet Gateway	Internet Connectivity
NAT Gateway	Outbound Internet Access
AWS IAM	Access Management

Project Architecture

Infrastructure Layer

- **Terraform** provisions the VPC, public and private subnets, route tables, security groups, IAM resources, the managed EC2 server, ECR, ECS, Application Load Balancer, and monitoring-related AWS resources.
- The application **VPC** contains two **public subnets** and two **private subnets** across separate Availability Zones.
- The public subnets provide internet connectivity through the **Internet Gateway** and host the internet-facing **Application Load Balancer** and **NAT Gateway**.
- The **ECS** application runs in the private subnets. The NAT Gateway provides outbound internet access when required.
- The **managed EC2 server** is placed in a public subnet so that the control node can establish SSH connectivity to it for configuration.

Control and Managed Node Architecture

- A dedicated **Control EC2** runs **Ansible**, **Jenkins**, **Terraform**, **AWS CLI**, **Docker**, and **Trivy**.
- A separate **Managed EC2** is configured by **Ansible** and hosts the DevOps services required by the project, including **SonarQube** and **Grafana**.
- The **Ansible control node** connects to the **managed node** through SSH using the managed node private key.
- **Jenkins** runs on the control node and executes the **Terraform** infrastructure pipeline and application **CI/CD pipeline**.

Infrastructure Pipeline

- **Terraform code** is stored in the project repository and processed by a dedicated Jenkins infrastructure pipeline.
- The pipeline performs **checkout, initialization, formatting/validation, planning, and application** of the Terraform configuration.

Application CI/CD Pipeline

Jenkins retrieves the application source, installs dependencies, lints and builds the React application, performs **SonarQube** analysis, builds the **Docker image**, scans it with **Trivy**, applies the security gate, and pushes the validated image to **Amazon ECR**.

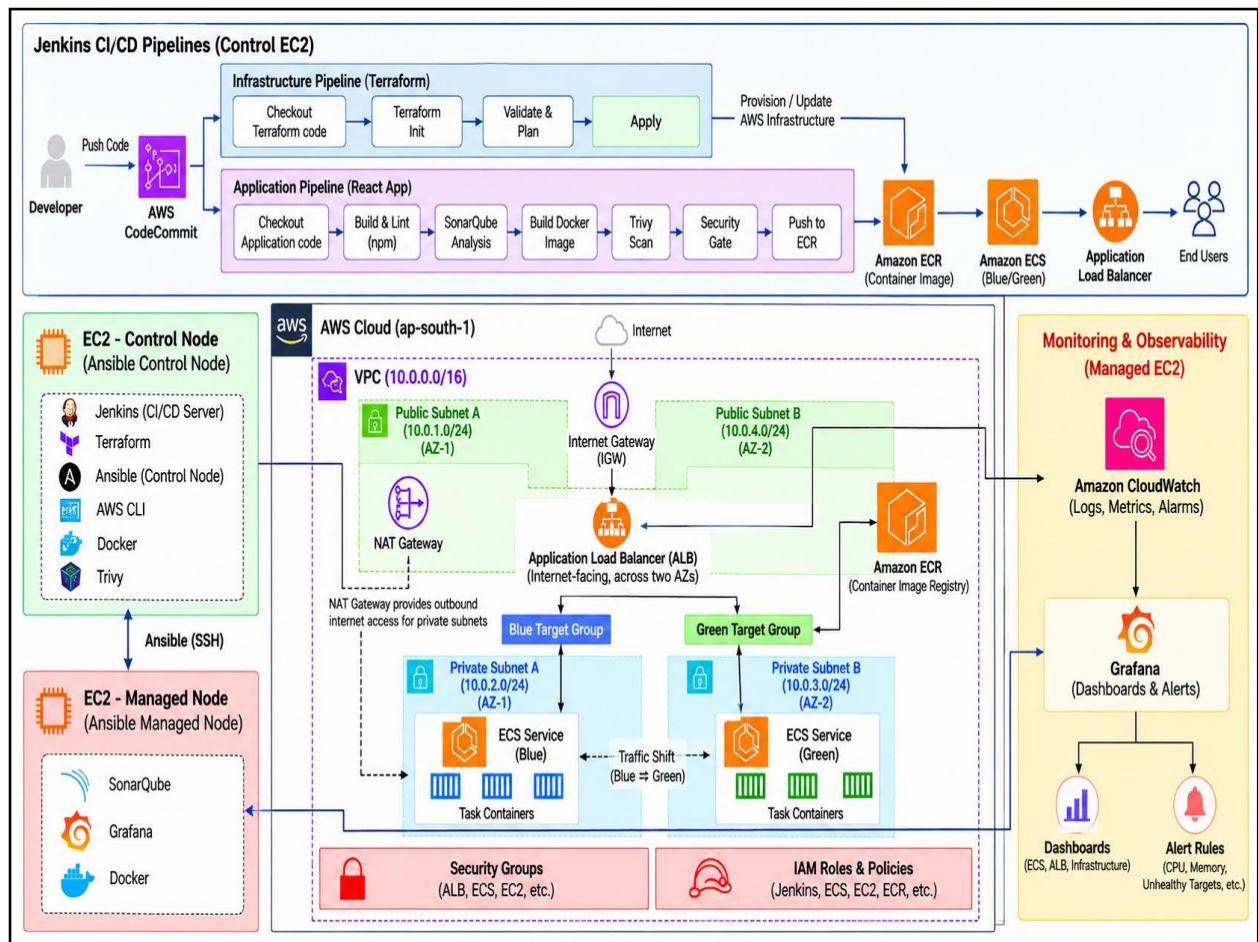
Application Deployment

- **Amazon ECS** runs the containerized application using **Fargate** in private subnets.
- **Blue and Green environments** use separate **ECS services** and **ALB target groups**.
- The **Green environment** is deployed and health-checked before traffic is switched from Blue to Green.
- The previous environment remains available during validation so that traffic can be returned to it if the new version fails validation.

Monitoring and Alerting

- **CloudWatch** collects application logs and ECS/ALB metrics.
- **Grafana** uses CloudWatch as a data source and presents dashboards for application and infrastructure monitoring.
- Grafana alert rules are configured for selected health conditions and tested by deliberately creating and then clearing the configured condition.

Architecture Diagram



Objectives

The main objectives are:

- Design an **AWS network** with **two public** and **two private subnets** across separate Availability Zones.
- Use **Terraform** to provision the project infrastructure through a **Jenkins Infrastructure Pipeline**.
- Use a dedicated **EC2 instance** as the **Ansible control node** and a separate EC2 instance as the **managed node**.
- Configure the managed node with Ansible and deploy the required DevOps services.
- Automate application validation, code analysis, container scanning, image creation, and **ECR publishing** through **Jenkins**.
- Use **SonarQube** and **Trivy** as security and quality checks in the application pipeline.
- Deploy the validated image to **ECS** using **Blue/Green deployment** and an

Application Load Balancer.

- Use **CloudWatch** for logs and metrics and **Grafana** for dashboards and alerting.
- Test **Grafana alert** triggering and recovery rather than only creating a dashboard.
- Maintain the application, Terraform, Ansible, Jenkins configuration, and documentation in a Git repository.

Prerequisites

- **AWS account** with permission to create the required resources.
- Initial **Control EC2** bootstrap instance with SSH access and sufficient resources for **Jenkins**, **Terraform**, **Ansible**, **Docker**, and the pipeline tools.
- **Git repository** containing the **application**, **Terraform configuration**, **Ansible playbooks**, and **Jenkins pipeline definitions**.
- **Ansible** installed on the **Control EC2**.
- **Terraform** and **AWS CLI** installed on the **Control EC2**.
- **Docker** installed and accessible to Jenkins on the Control EC2.
- A **React + Vite Quiz App** with package.json and package-lock.json.
- An **AWS CodeCommit** repository or the selected Git repository available to Jenkins.
- Appropriate **IAM permissions** for the Control EC2 and ECS task execution role.
- A key pair that allows the Control EC2 to connect to the Managed EC2.

☐	Name 	Instance ID	Instance state 	Instance type 	Status check
☐	capstone-cont...	i-07d22fd3243699b3b	 Running  	m7i-flex.large	 3/3 checks passed

```
root@ip-172-31-11-184:/home/ubuntu# git --version
git version 2.53.0
root@ip-172-31-11-184:/home/ubuntu# docker --version
Docker version 29.8.2, build 7fc2dfff
root@ip-172-31-11-184:/home/ubuntu# aws --version
aws-cli/2.37.10 Python/3.14.6 Linux/7.0.0-1006-aws script-exe/x86_64.ubuntu.26
root@ip-172-31-11-184:/home/ubuntu# terraform version
Terraform v1.16.5
on linux_amd64
root@ip-172-31-11-184:/home/ubuntu# ansible --version
ansible [core 2.21.5]
  config file = /etc/ansible/ansible.cfg
  configured module search path = ['/root/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /root/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.14.4 (main, Aug 20 2026, 10:41:58) [GCC 15.2.0] (/usr/bin/python3)
  jinja version = 3.1.6
  pyyaml version = 6.0.3 (with libyaml v0.2.5)
root@ip-172-31-11-184:/home/ubuntu# jenkins --version
2.580.1
root@ip-172-31-11-184:/home/ubuntu# trivy --version
Version: 0.75.0
```

Note: The Control EC2 is treated as a bootstrap resource because Jenkins must exist before it can execute the Terraform infrastructure pipeline. The remaining project infrastructure is then provisioned and managed through the Terraform pipeline.

Step 1: Application Containerization with Docker

The **Quiz App**, developed using React and Vite, is selected as the application. The application source code is available in the project repository below.

[Quiz App](#)

The application is containerized using Docker.

1.1 Create the Dockerfile

Create a Dockerfile in the root directory of the Quiz App. A multi-stage Docker build is used to build the React + Vite application and serve the generated production files using Nginx.

Dockerfile

```
FROM node:22-alpine AS builder

WORKDIR /app

COPY package*.json ./

RUN npm ci

COPY . .

RUN npm run build

FROM nginx:alpine

RUN apk update && apk upgrade --no-cache

COPY --from=builder /app/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

The first stage installs the application dependencies and generates the production build using Vite. The second stage uses Nginx to serve the generated static files, resulting in a lightweight

production container.

1.2 Build and Verify the Docker Image

Build the Docker image from the root directory of the Quiz App.

```
$ docker build -t quiz-app .
```

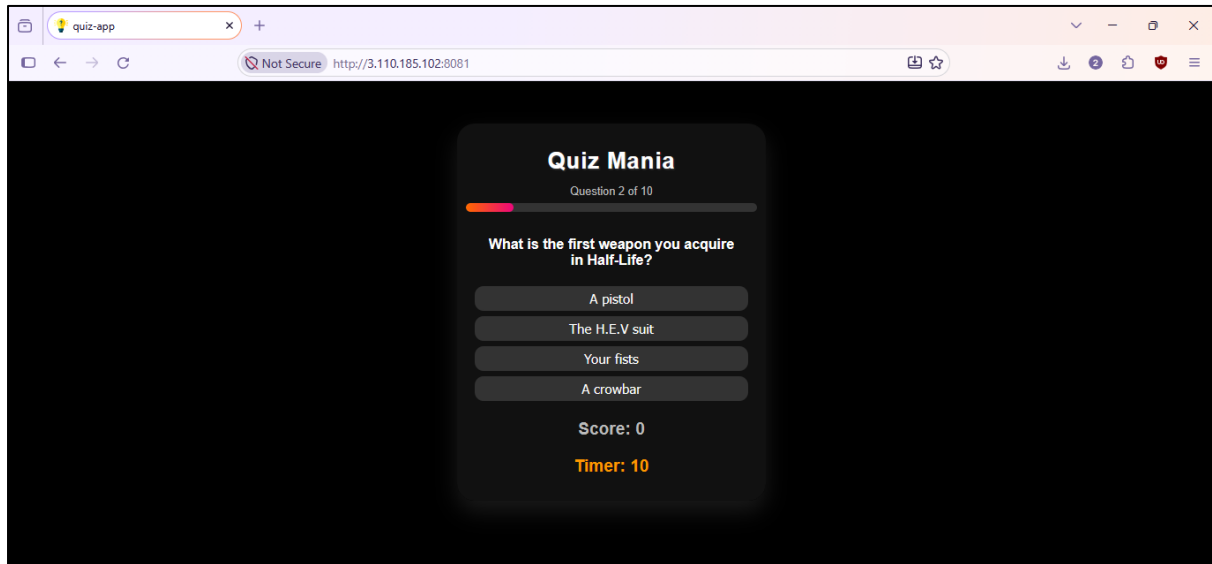
```
[+] Building 12.7s (14/14) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 258B                                0.0s
=> [internal] load metadata for docker.io/library/node:22-alpine  2.2s
=> [internal] load metadata for docker.io/library/nginx:alpine    2.2s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
```

Run the container locally to verify that the application is served correctly.

```
$ docker run -d -p 8081:80 --name quiz-app quiz-app
```

Open the application in a browser.

```
http://<CONTROL_EC2_PUBLIC_IP>:8081
```



Step 2: Core Infrastructure Provisioning with Terraform

Terraform is used to provision the core AWS infrastructure required for the application. The Control EC2 is treated as a bootstrap resource because Jenkins must be available before the infrastructure pipeline can be executed. The remaining AWS infrastructure is provisioned and managed through Terraform.

2.1 Create the Terraform Project

Create a Terraform project and organize the configuration into separate .tf files based on the resources being provisioned.

The project structure is as follows:-

```
terraform/  
├─ provider.tf  
├─ variables.tf  
├─ vpc.tf  
├─ subnet.tf  
├─ route_table.tf  
├─ security_group.tf  
├─ iam.tf  
├─ ec2.tf  
├─ ecr.tf  
├─ ecs.tf  
├─ alb.tf  
├─ outputs.tf  
└─ terraform.tfvars
```

The Terraform configuration provisions the networking, security, compute, container registry, and ECS resources required by the project.

2.2 Configure the AWS Provider

The AWS provider is configured to allow Terraform to create and manage resources in the selected AWS region.

provider.tf

```
terraform {  
  required_version = ">= 1.5.0"  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 6.0"  
    }  
  }  
}
```



```
provider "aws" {  
    region = var.aws_region  
}
```

2.3 Define Terraform Variables

The infrastructure uses variables for values that may differ between AWS accounts or environments, such as the AWS region, Availability Zones, and network CIDR ranges.

variables.tf

```
variable "aws_region" {  
    description = "AWS region"  
    type        = string  
    default     = "<AWS_REGION>"  
}  
  
variable "project_name" {  
    description = "Project name"  
    type        = string  
    default     = "capstone"  
}  
  
variable "vpc_cidr" {  
    description = "VPC CIDR block"  
    type        = string  
    default     = "10.0.0.0/16"  
}  
  
variable "public_subnet_cidrs" {  
    description = "Public subnet CIDR blocks"  
    type        = list(string)  
    default = [  
        "10.0.1.0/24",  
        "10.0.2.0/24"  
    ]  
}
```

```

variable "private_subnet_cidrs" {
  description = "Private subnet CIDR blocks"
  type       = list(string)
  default = [
    "10.0.3.0/24",
    "10.0.4.0/24"
  ]
}

variable "availability_zones" {
  description = "Availability Zones"
  type       = list(string)

  default = [
    "<AWS_REGION>a",
    "<AWS_REGION>b"
  ]
}

variable "admin_cidr" {
  description = "CIDR allowed for administrative access"
  type       = string
  default    = "<CONTROL_EC2_PUBLIC_IP>/32"
}

```

2.4 Provide Environment-Specific Values

Environment-specific values can be supplied through the Terraform variables file.

terraform.tfvars

```

aws_region = "<AWS_REGION>"
project_name = "capstone"
vpc_cidr = "10.0.0.0/16"
instance_type = "m7i-flex.large"
public_subnet_cidrs = [
  "10.0.1.0/24",
  "10.0.2.0/24"
]

```

```

private_subnet_cidrs = [
    "10.0.3.0/24",
    "10.0.4.0/24"
]

availability_zones = [
    "<AWS_REGION>a",
    "<AWS_REGION>b"
]

admin_cidr = "<CONTROL_EC2_PUBLIC_IP>/32"

```

2.5 Provision the VPC

Terraform creates an isolated VPC for the application infrastructure. DNS support and DNS hostnames are enabled so that AWS resources can communicate using DNS where required.

vpc.tf

```

resource "aws_vpc" "main" {
  cidr_block      = var.vpc_cidr
  enable_dns_support = true
  enable_dns_hostnames = true

  tags = {
    Name = "${var.project_name}-vpc"
  }
}

```

The VPC is divided into **two public subnets across separate Availability Zones and two private subnets across separate Availability Zones**. The public subnets are used for the internet-facing Application Load Balancer, the Managed EC2/DevOps server, and the NAT Gateway. The private subnets are used for the ECS application workloads.

subnet.tf

```

resource "aws_subnet" "public" {
  count = length(var.public_subnet_cidrs)

  vpc_id      = aws_vpc.main.id
  cidr_block   = var.public_subnet_cidrs[count.index]
  availability_zone = var.availability_zones[count.index]
  map_public_ip_on_launch = true
}

```

```

resource "aws_subnet" "public" {
  count = length(var.public_subnet_cidrs)
  vpc_id      = aws_vpc.main.id
  cidr_block   = var.public_subnet_cidrs[count.index]
  availability_zone = var.availability_zones[count.index]
  map_public_ip_on_launch = true
  tags = {
    Name = "${var.project_name}-public-${count.index + 1}"
  }
}

resource "aws_subnet" "private" {
  count = length(var.private_subnet_cidrs)
  vpc_id      = aws_vpc.main.id
  cidr_block   = var.private_subnet_cidrs[count.index]
  availability_zone = var.availability_zones[count.index]
  tags = {
    Name = "${var.project_name}-private-${count.index + 1}"
  }
}

```

2.6 Create the Internet Gateway

The Internet Gateway provides internet connectivity for the public subnet infrastructure, including the Application Load Balancer and Managed EC2.

internet_gateway.tf

```

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
  tags = {
    Name = "${var.project_name}-igw"
  }
}

```

2.7 Create the NAT Gateway

The NAT Gateway is placed in a **public subnet** and provides outbound internet access to the ECS workloads running in the **private subnets** when required.

nat_gateway.tf

```
resource "aws_eip" "nat" {
  domain = "vpc"
  tags = {
    Name = "${var.project_name}-nat-eip"
  }
}

resource "aws_nat_gateway" "main" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public[0].id
  depends_on = [
    aws_internet_gateway.main
  ]
  tags = {
    Name = "${var.project_name}-nat"
  }
}
```

2.8 Configure Route Tables

The **public route table** sends internet-bound traffic through the Internet Gateway. The **private route table** sends outbound internet traffic through the NAT Gateway.

route_table.tf

```
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }
  tags = {
    Name = "${var.project_name}-public-rt"
  }
}
```

```

resource "aws_route_table_association" "public" {
  count = length(aws_subnet.public)
  subnet_id    = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table" "private" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block    = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.main.id
  }

  tags = {
    Name = "${var.project_name}-private-rt"
  }
}

resource "aws_route_table_association" "private" {
  count = length(aws_subnet.private)
  subnet_id    = aws_subnet.private[count.index].id
  route_table_id = aws_route_table.private.id
}

```

2.9 Configure Security Groups

The Managed EC2 requires SSH access from the Control EC2 because the Control EC2 acts as the **Ansible control node** and the Managed EC2 acts as the **Ansible managed node**.

security_group.tf

```

resource "aws_security_group" "managed_ec2" {
  name    = "${var.project_name}-managed-ec2-sg"
  vpc_id = aws_vpc.main.id

  ingress {
    description = "SSH from Control EC2"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
  }
}

```

```

    cidr_blocks = [var.admin_cidr]
  }
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = {
    Name = "${var.project_name}-managed-ec2-sg"
  }
}

```

2.10 Configure IAM

The ECS task execution role gives ECS permission to perform tasks such as pulling the application's container image from Amazon ECR and sending container logs to CloudWatch.

iam.tf

```

resource "aws_iam_role" "ecs_execution" {
  name = "${var.project_name}-ecs-execution-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = {
        Service = "ecs-tasks.amazonaws.com"
      }
      Action = "sts:AssumeRole"
    }]
  })
}

resource "aws_iam_role_policy_attachment" "ecs_execution" {
  role      = aws_iam_role.ecs_execution.name
  policy_arn = "arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy"
}

```

2.11 Initialize and validate Terraform

Validate the configuration

```
$ terraform validate
```

```
Success! The configuration is valid.
```

Review the execution

```
$ terraform plan
```

Step 3: Application Infrastructure Provisioning with Terraform

The **Managed EC2** is created as the Ansible managed node, **Amazon ECR** is created to store the application's Docker image, and **Amazon ECS** is prepared to run the application using Fargate. An **Application Load Balancer** is placed across the public subnets to receive requests from users and forward them to ECS tasks running in the private subnets. Separate **Blue and Green target groups** are also created to support the Blue/Green deployment.

3.1 Provision the Managed EC2

The managed EC2 instance acts as the **Ansible managed node** and will host Docker, SonarQube, and Grafana. It is placed in a public subnet so that the Control EC2 can connect to it through SSH.

variables.tf

```
variable "ami_id" {
    description = "AMI ID for the Managed EC2"
    type      = string
    default   = "<AMI_ID>"
}

variable "instance_type" {
    description = "EC2 instance type"
    type       = string
    default    = "m7i-flex.large"
}

variable "key_name" {
    description = "EC2 key pair name"
    type       = string
    default    = "<EC2_KEY_PAIR_NAME>"
}
```


Create the Managed EC2 instance.

ec2.tf

```
resource "aws_instance" "managed" {  
    ami      = var.ami_id  
    instance_type = var.instance_type  
    subnet_id  = aws_subnet.public[0].id  
    key_name = var.key_name  
    vpc_security_group_ids = [  
        aws_security_group.managed_ec2.id  
    ]  
    associate_public_ip_address = true  
    root_block_device {  
        volume_size = 20  
        volume_type = "gp3"  
    }  
    tags = {  
        Name = "${var.project_name}-managed-ec2"  
    }  
}
```

The instance is placed in the first public subnet and receives a public IP address. This allows the Control EC2 to establish the SSH connection required by Ansible.

Configure access to SonarQube and Grafana.

security_group.tf

```
ingress {  
    description = "SonarQube"  
    from_port   = 9000  
    to_port     = 9000  
    protocol    = "tcp"  
    cidr_blocks = [var.admin_cidr]  
}  
  
ingress {  
    description = "Grafana"  
    from_port   = 3000
```

```
to_port    = 3000

protocol   = "tcp"

cidr_blocks = [var.admin_cidr]

}
```

3.2 Create ECR, ECS, and CloudWatch Resources

Create the Amazon ECR Repository.

Amazon ECR stores the Docker image produced by the Jenkins Application Pipeline. After the image passes the required analysis and security checks, Jenkins pushes it to this repository.

ecr.tf

```
resource "aws_ecr_repository" "app" {
  name          = "${var.project_name}-app"
  image_tag_mutability = "MUTABLE"
  image_scanning_configuration {
    scan_on_push = true
  }
  tags = {
    Name = "${var.project_name}-ecr"
  }
}
```

Create the ECS Cluster

Amazon ECS provides the environment in which the Quiz App will run using AWS Fargate. Fargate removes the need to manage EC2 instances for the application containers.

ecs.tf

```
resource "aws_ecs_cluster" "main" {
  name = "${var.project_name}-cluster"
  tags = {
    Name = "${var.project_name}-cluster"
  }
}
```

Create the CloudWatch Log Group

CloudWatch collects the logs generated by the ECS application containers.

ecs.tf

```
resource "aws_cloudwatch_log_group" "ecs" {  
  name      = "/ecs/${var.project_name}"  
  retention_in_days = 7  
}
```

3.3 Configure ECS and Load Balancer Networking

The ECS tasks run in the private subnets and should receive application traffic through the Application Load Balancer rather than directly from the internet.

Create the ECS Security Group

security_group.tf

```
resource "aws_security_group" "ecs" {  
  name = "${var.project_name}-ecs-sg"  
  vpc_id = aws_vpc.main.id  
  ingress {  
    description = "HTTP from Application Load Balancer"  
    from_port = 80  
    to_port = 80  
    protocol = "tcp"  
    security_groups = [aws_security_group.alb.id]  
  }  
  egress {  
    from_port = 0  
    to_port = 0  
    protocol = "-1"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  tags = {  
    Name = "${var.project_name}-ecs-sg"  
  }  
}
```

Create the ALB Security Group and Load Balancer

The Application Load Balancer is the public entry point for the Quiz App. It is placed across both public subnets and forwards requests to ECS tasks in the private subnets.

alb.tf

```
resource "aws_security_group" "alb" {
  name = "${var.project_name}-alb-sg"
  vpc_id = aws_vpc.main.id
  ingress {
    description = "HTTP from Internet"
    from_port = 80
    to_port = 80
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port = 0
    to_port = 0
    protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = {
    Name = "${var.project_name}-alb-sg"
  }
}

resource "aws_lb" "app" {
  name = "${var.project_name}-alb"
  internal = false
  load_balancer_type = "application"
  subnets = aws_subnet.public[*].id
  security_groups = [
    aws_security_group.alb.id
  ]
  tags = {
    Name = "${var.project_name}-alb"
  }
}
```

3.4 Configure Blue/Green Target Group

Blue and green target groups are created for the ALB

alb.tf

```
resource "aws_lb_target_group" "blue" {
  name      = "${var.project_name}-blue"
  port      = 80
  protocol  = "HTTP"
  target_type = "ip"
  vpc_id    = aws_vpc.main.id
  health_check {
    path          = "/"
    protocol      = "HTTP"
    port          = "traffic-port"
    healthy_threshold = 2
    unhealthy_threshold = 2
    timeout       = 5
    interval      = 30
  }
  tags = {
    Name = "${var.project_name}-blue-target-group"
  }
}

resource "aws_lb_target_group" "green" {
  name      = "${var.project_name}-green"
  port      = 80
  protocol  = "HTTP"
  target_type = "ip"
  vpc_id    = aws_vpc.main.id
  health_check {
    path          = "/"
    protocol      = "HTTP"
    port          = "traffic-port"
    healthy_threshold = 2
  }
}
```

```

    unhealthy_threshold = 2

    timeout      = 5

    interval     = 30
  }

  tags = {
    Name = "${var.project_name}-green-target-group"
  }
}

```

The / health check verifies that the Nginx-served Quiz App is responding correctly.

3.5 Configure the ALB Listener

The ALB listener receives HTTP requests on port 80. The Blue target group is configured as the initial default target group.

alb.tf

```

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.app.arn

  port      = 80
  protocol  = "HTTP"

  default_action {
    type = "forward"

    target_group_arn = aws_lb_target_group.blue.arn
  }
}

```

3.6 Define Terraform Outputs

outputs.tf

```

output "managed_ec2_public_ip" {
  description = "Public IP address of the Managed EC2"

  value = aws_instance.managed.public_ip
}

output "managed_ec2_private_ip" {
  description = "Private IP address of the Managed EC2"

  value = aws_instance.managed.private_ip
}

```

```

output "ecr_repository_url" {
  description = "ECR repository URL"
  value      = aws_ecr_repository.app.repository_url
}

output "ecs_cluster_name" {
  description = "ECS cluster name"
  value      = aws_ecs_cluster.main.name
}

output "alb_dns_name" {
  description = "Application Load Balancer DNS name"
  value      = aws_lb.app.dns_name
}

```

3.7 Create the Jenkins Infrastructure Pipeline

Jenkins runs on the Control EC2 and uses the instance's IAM role to access AWS services and AWS CodeCommit without storing long-lived AWS access keys. The Infrastructure Pipeline is configured to use the quiz-app CodeCommit repository as its source and executes Terraform from the terraform/ directory.

Jenkinsfile-infrastructure

```

pipeline {
  agent any
  environment {
    AWS_REGION = 'ap-south-1'
  }
  stages {
    stage('Checkout Terraform Code') {
      steps {
        checkout scm
      }
    }
    stage('Terraform Init') {
      steps {
        dir('terraform') {

```

```

        sh '''
            pwd
            ls -la
            terraform init
        '''
    }
}

stage('Validate & Plan') {
    steps {
        dir('terraform') {
            sh '''
                terraform fmt -check
                terraform validate
                terraform plan
            '''
        }
    }
}

stage('Apply') {
    steps {
        input message: 'Approve Terraform infrastructure deployment?'
        dir('terraform') {
            sh 'terraform apply -auto-approve'
        }
    }
}
}

```

3.8 Run and Verify the Infrastructure Pipeline

Jenkins runs on the Control EC2 and uses the instance's IAM role to access AWS services and AWS CodeCommit without storing long-lived AWS access keys. The Infrastructure Pipeline is

configured to use the quiz-app CodeCommit repository as its source and executes Terraform from the terraform/ directory.

☐ capstone-man...i-0873f5e6cd7356f84Runningm7i-flex.large3/3 checks passed

Outputs:

```
[emaib_dns_name = "xxx.ap-south-1.elb.amazonaws.com"
ecr_repository_url = "xxx.dkr.ecr.ap-south-1.amazonaws.com/capstone-app"
ecs_cluster_name = "capstone-cluster"
managed_ec2_private_ip = "10.0.1.xxx"
managed_ec2_public_ip = "xxx.xxx.xxx.xxx"
[Pipeline] }
[Pipeline] // dir
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Step 4: Server Configuration with Ansible

The **Control EC2** is used as the Ansible control node to connect to the **Managed EC2** through SSH and configure the server.

4.1 Configure the Ansible Control Node

The Ansible inventory identifies the Managed EC2 and defines the SSH details required for the Control EC2 to connect to it.

inventory.ini

```
[managed]

managed-ec2 ansible_host=<MANAGED_EC2_PUBLIC_IP> ansible_user=ubuntu

[all:vars]

ansible_ssh_private_key_file=<MANAGED_INSTANCE_KEY_PAIR_LOCATION>
```

Replace <MANAGED_EC2_PUBLIC_IP> with the public IP address of the Managed EC2.

The managed group represents the server that Ansible will configure. The ansible_user specifies the Ubuntu user used to connect to the instance.

4.2 Create Ansible Configuration

The Ansible configuration file defines the default settings used when executing the playbook.

ansible.cfg

```
[defaults]

inventory = ./inventory.ini

host_key_checking = False
```

Create the Ansible project structure on the Control EC2.

```
ansible/
├── inventory
├── ansible.cfg
├── site.yml
├── roles/
│   ├── docker/
│   │   └── tasks/
│   │       └── main.yml
│   ├── sonarqube/
│   │   ├── tasks/
│   │   └── main.yml
│   └── grafana/
│       ├── tasks/
│       └── main.yml
```

The site.yml file acts as the main playbook and calls the roles used to configure the Managed EC2.

site.yml

```
---

- name: Configure Managed EC2

  hosts: managed

  become: true

  roles:

    - docker

    - sonarqube

    - grafana
```

The `become: true` option allows Ansible to perform administrative operations on the Managed EC2.

The Docker modules used by the SonarQube and Grafana roles require the Ansible Docker collection. Install it on the Control EC2.

```
$ ansible-galaxy collection install community.docker
```

4.4 Configure Managed EC2

Docker Configuration

roles/docker/tasks/main.yml

```
---
- name: Update package cache
  apt:
    update_cache: yes
- name: Install Docker
  apt:
    name: docker.io
    state: present
- name: Start Docker service
  service:
    name: docker
    state: started
    enabled: true
```

SonarQube Configuration

SonarQube is deployed as a Docker container on port 9000.

roles/sonarqube/tasks/main.yml

```
---
- name: Pull SonarQube image
  community.docker.docker_image:
    name: sonarqube:community
    source: pull
```

```
- name: Run SonarQube container
  community.docker.docker_container:
    name: sonarqube
    image: sonarqube:community
    state: started
    restart_policy: unless-stopped
    ports:
      - "9000:9000"
```

Grafana Configuration

Grafana is deployed as a Docker container on port 3000.

roles/grafana/tasks/main.yml

```
---
- name: Pull Grafana image
  community.docker.docker_image:
    name: grafana/grafana:latest
    source: pull
- name: Run Grafana container
  community.docker.docker_container:
    name: grafana
    image: grafana/grafana:latest
    state: started
    restart_policy: unless-stopped
    ports:
      - "3000:3000"
```

4.5 Execute the Ansible Playbook

Verify that the Control EC2 can connect to the Managed EC2 through Ansible.

```
$ ansible managed -m ping
```

```
managed-ec2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.14"
  },
  "changed": false,
  "ping": "pong"
}
```

Execute the playbook.

```
$ ansible-playbook site.yml
```

```
PLAY RECAP *****
managed-ec2          : ok=8    changed=6    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Verify the Docker installation.

```
$ docker --version
```

```
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
```

Verify the running containers.

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
602864c01b72	grafana/grafana:latest	"/run.sh"	About a minute ago	Up About a minute	0.0.0.0:3000->3000/tcp
af5c973c8f09	sonarqube:community	"/opt/sonarqube/dock..."	2 minutes ago	Up 2 minutes	0.0.0.0:9000->9000/tcp

Verify SonarQube.

```
http://<MANAGED_EC2_PUBLIC_IP>:9000
```

SonarQube community

Projects Issues Rules Quality profiles Quality gates Administration More

How do you want to create your project?

Do you want to benefit from all of SonarQube Community Build's features (like repository import and Pull Request decoration)?

Create your project from your favorite DevOps platform.

First, you need to set up a DevOps platform configuration.

- Import from Azure DevOps **Setup**
- Import from Bitbucket Cloud **Setup**
- Import from Bitbucket Data Center **Set...**
- Import from GitHub **Setup**
- Import from GitLab

Are you just testing or have an advanced use-case?

[Create a local project](#)

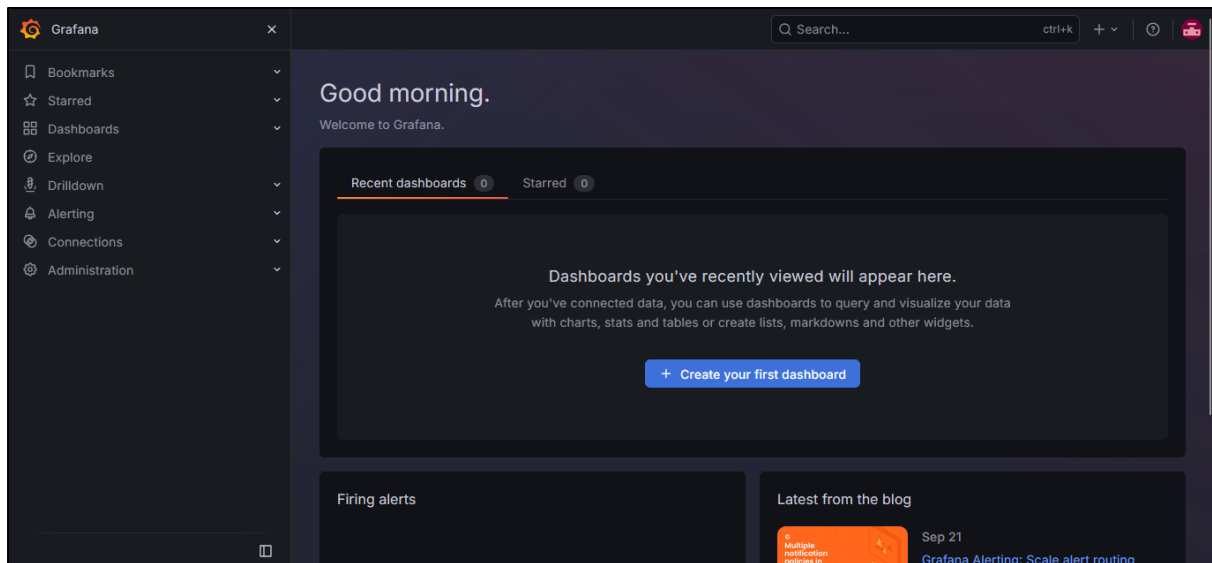
Get the most out of SonarQube Community Build!

Take advantage of the whole ecosystem by using SonarQube for IDE, a free IDE plugin that helps you find and fix issues earlier in your workflow. Connect SonarQube for IDE to SonarQube Community Build to sync rule sets and issue states.

[Learn More](#) [Dismiss](#)

Verify Grafana.

```
http://<MANAGED_EC2_PUBLIC_IP>:3000
```

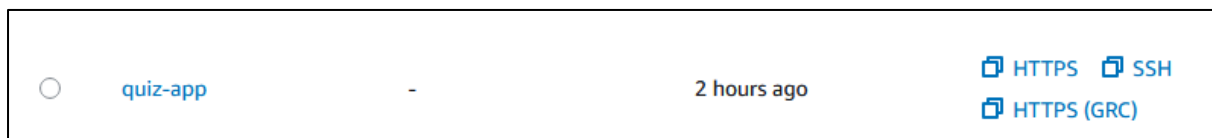


Step 5: Jenkins CI/CD Pipeline

Jenkins runs on the **Control EC2** and retrieves the React + Vite Quiz App source code from AWS CodeCommit. The pipeline installs the application dependencies, validates and builds the application, performs code analysis using SonarQube running on the **Managed EC2**, builds the Docker image, scans the image using Trivy, applies the security gate, and pushes the validated image to Amazon ECR.

5.1 Configure the CodeCommit Repository

AWS CodeCommit is used as the source-code repository for the Jenkins application pipeline. The React + Vite Quiz App is placed in the CodeCommit repository along with the files required to build and containerize the application.



5.2 Configure Jenkins to Access CodeCommit

Verify that the AWS CLI on the Control EC2 can access the AWS account.

```
$ aws sts get-caller-identity
```

Verify that the CodeCommit repository is accessible.

```
$ aws codecommit list-repositories --region <AWS_REGION>
```

5.3 Create the Jenkins Pipeline

Create a Jenkinsfile in the root directory of the application repository.

Jenkinsfile

```
pipeline {
  agent any
  environment {
    AWS_REGION    = '<AWS_REGION>'
    AWS_ACCOUNT_ID = '<AWS_ACCOUNT_ID>'
    ECR_REPOSITORY = 'capstone-app'
    IMAGE_TAG      = "${BUILD_NUMBER}"
    ECR_REGISTRY   = "${AWS_ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com"
  }
  stages {
    stage('Checkout') {
      steps {
        git branch: 'master',
            url: '<CODECOMMIT_REPOSITORY_URL>'
      }
    }
    stage('Install Dependencies') {
      steps {
        sh 'npm ci'
      }
    }
    stage('Lint') {
      steps {
        sh 'npm run lint'
      }
    }
  }
}
```

```
stage('Build Application') {
    steps {
        sh 'npm run build'
    }
}

stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv('SonarQube') {
            sh '''
                sonar-scanner \
                -Dsonar.projectKey=quiz-app \
                -Dsonar.sources=src
            '''
        }
    }
}

stage('Build Docker Image') {
    steps {
        sh "docker build -t ${ECR_REPOSITORY}:${IMAGE_TAG} ."
    }
}

stage('Trivy Scan') {
    steps {
        sh """
            trivy image \
            --exit-code 1 \
            --severity HIGH,CRITICAL \
            ${ECR_REPOSITORY}:${IMAGE_TAG}
        """
    }
}
```



```

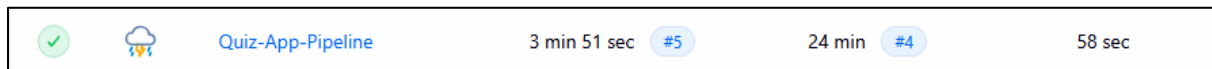
stage('Security Gate') {
    steps {
        timeout(time: 5, unit: 'MINUTES') {
            waitForQualityGate abortPipeline: true
        }
    }
}

stage('Login to ECR') {
    steps {
        sh '''
            aws ecr get-login-password \
            --region ${AWS_REGION} |
            docker login \
            --username AWS \
            --password-stdin ${ECR_REGISTRY}
        '''
    }
}

stage('Push to ECR') {
    steps {
        sh '''
            docker tag ${ECR_REPOSITORY}:${IMAGE_TAG} \
            ${ECR_REGISTRY}/${ECR_REPOSITORY}:${IMAGE_TAG}
            docker push \
            ${ECR_REGISTRY}/${ECR_REPOSITORY}:${IMAGE_TAG}
        '''
    }
}
}

```

The pipeline uses the Jenkins `IMAGE_TAG` as the Docker image tag so that each pipeline execution produces a uniquely identifiable image.



5.4 Build and Validate the Application

The pipeline installs the application dependencies and performs the required validation before creating the Docker image.

```
$ npm ci
$ npm run lint
$ npm run build
```

The `npm ci` command installs the dependencies defined in `package-lock.json`. The lint stage checks the source code for errors and the build stage generates the production-ready Vite build.

5.5 Integrate SonarQube Code Analysis

SonarQube is integrated into Jenkins to analyze the application source code for code quality and security issues.

First, the SonarScanner CLI is configured on the **Control EC2**, where Jenkins executes the application pipeline. Verify the installation.

```
$ sonar-scanner --version
```

Create a **User Token** in SonarQube from:

Profile → My Account → Security → Tokens

Generate a token and store it securely in Jenkins as a **Secret text** credential.

In Jenkins, go to:

Manage Jenkins → Credentials → Add Credentials

Configure:

```
Kind: Secret text
Secret: <SONARQUBE_TOKEN>
ID: sonarqube-token
```

Configure the SonarQube server in Jenkins under:

Manage Jenkins → System → SonarQube servers

Name: SonarQube
Server URL: http://<MANAGED_EC2_PUBLIC_IP>:9000
Authentication Token: sonarqube-token

Configure a webhook in SonarQube under:

Administration → Configuration → Webhooks

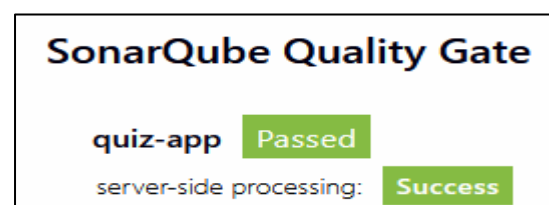
Name: Jenkins
URL: http://<CONTROL_EC2_PUBLIC_IP>:8080/sonarqube-webhook/

The webhook allows SonarQube to return the Quality Gate result to Jenkins. The trailing / in /sonarqube-webhook/ should be included.

The pipeline then executes.

```
stage('SonarQube Analysis') {  
  steps {  
    withSonarQubeEnv('SonarQube') {  
      sh '''  
        sonar-scanner \  
        -Dsonar.projectKey=quiz-app \  
        -Dsonar.sources=src  
      '''  
    }  
  }  
}
```

The analysis result is sent to the SonarQube server for evaluation.



5.6 Build the Docker Image

After the application passes the dependency installation, linting, and build stages, Jenkins builds the Docker image using the **Dockerfile**.

```
$ docker build -t capstone-app:${BUILD_NUMBER} .
```

The image is tagged using the Jenkins build number.

Verify the generated image.

```
$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
XXXXX.dkr.ecr.ap-south-1.amazonaws.com/capstone-app:5	486d1e04075d	113MB	32.9MB	

5.7 Scan the Docker Image with Trivy

Trivy scans the generated Docker image for known vulnerabilities.

```
trivy image \  
--exit-code 1 \  
--severity HIGH,CRITICAL \  
${ECR_REPOSITORY}:${IMAGE_TAG}
```

The configured policy causes the pipeline to stop when HIGH or CRITICAL vulnerabilities are detected.

If no vulnerabilities meeting the configured severity threshold are detected, the pipeline continues to the security gate.

Target	Type	Vulnerabilities	Secrets
capstone-app:5 (alpine 3.24.2)	alpine	0	-

5.8 Configure the Security Gate

The security gate ensures that the application proceeds to Amazon ECR only after the required security checks have passed.

The Jenkins pipeline waits for the SonarQube quality gate result.

```
stage('Security Gate') {  
    steps {  
        timeout(time: 5, unit: 'MINUTES') {  
            waitForQualityGate abortPipeline: true  
        }  
    }  
}
```

```
SonarQube task 'eef27235-dbd5-4155-bcba-44ced469ee96' completed. Quality gate is 'OK'
```

If the SonarQube quality gate fails, Jenkins stops the pipeline.

A HIGH or CRITICAL vulnerability causes the Trivy command to return a failure status, which stops the Jenkins pipeline.

5.9 Push the Validated Image to Amazon ECR

After the application passes the required validation and security checks, Jenkins authenticates with Amazon ECR.

```
aws ecr get-login-password \  
--region ${AWS_REGION} |  
docker login \  
--username AWS \  
--password-stdin ${ECR_REGISTRY}
```

Tag the Docker image with the ECR repository URI.

```
docker tag ${ECR_REPOSITORY}:${IMAGE_TAG} \  
${ECR_REGISTRY}/${ECR_REPOSITORY}:${IMAGE_TAG}
```

Push the image to Amazon ECR.

```
docker push \  
${ECR_REGISTRY}/${ECR_REPOSITORY}:${IMAGE_TAG}
```

Verify that the image is available in the ECR repository.

```
aws ecr describe-images \  
--repository-name capstone-app \  
--region <AWS_REGION>
```

```
"repositoryName": "capstone-app",
"imageDigest": "sha256:486d1e04075db9f61fa72c512a8a63379ec8db62ba928f5dd0f3d42c5b4e7b82",
"imageTags": [
  "5"
```

The validated Docker image is now stored in **Amazon ECR** and is ready to be deployed to **Amazon ECS**.

Step 6: Application Deployment to Amazon ECS

The validated Docker image stored in Amazon ECR is deployed to Amazon ECS using a Blue/Green deployment approach. The Blue and Green environments are connected to separate Application Load Balancer target groups. Jenkins deploys the new application version to the Green environment, validates its health, and then shifts the Application Load Balancer traffic from Blue to Green. The Blue environment remains available so that traffic can be returned to the previous version if required.

6.1 Create the ECS Task Definition

An ECS task definition specifies how the Quiz App container runs on AWS Fargate. Since the Docker image was tagged using the Jenkins build number, the task definition uses the same image tag. This ensures that the exact image that passed the CI/CD pipeline is deployed.

ecs-task-definition.json

```
{
  "family": "quiz-app",
  "networkMode": "awsvpc",
  "requiresCompatibilities": [
    "FARGATE"
  ],
  "cpu": "256",
  "memory": "512",
  "executionRoleArn": "<ECS_TASK_EXECUTION_ROLE_ARN>",
  "containerDefinitions": [
    {
      "name": "quiz-app",
      "image": "<AWS_ACCOUNT_ID>.dkr.ecr.<AWS_REGION>.amazonaws.com/capstone-app:<IMAGE_TAG>",
      "essential": true,
      "portMappings": [
        {
          "containerPort": 80,
```

```

    "protocol": "tcp"
  }
],
"logConfiguration": {
  "logDriver": "awslogs",
  "options": {
    "awslogs-group": "/ecs/capstone",
    "awslogs-region": "<AWS_REGION>",
    "awslogs-stream-prefix": "ecs"
  }
}
}
}
}
}

```

Replace:

- <AWS_ACCOUNT_ID> with the AWS account ID.
- <AWS_REGION> with the AWS region.
- <IMAGE_TAG> with the Jenkins build number.
- <ECS_TASK_EXECUTION_ROLE_ARN> with the ECS task execution role ARN.

The task uses the awsvpc network mode because the Fargate tasks run inside the private subnets. The container listens on port 80 and receives traffic through the Application Load Balancer.

6.2 Configure Blue and Green ECS Services

Two ECS services are used for the Blue/Green deployment. The **Blue service** represents the currently available application environment, while the **Green service** is used to deploy and validate the new application version.

Create the Blue Service

```

aws ecs create-service \
  --cluster capstone-cluster \
  --service-name quiz-app-blue \
  --task-definition quiz-app \
  --desired-count 1 \
  --launch-type FARGATE \
  --network-configuration \
    "awsvpcConfiguration={subnets=[<PRIVATE_SUBNET_A>,<PRIVATE_SUBNET_B>],securityGroups=[<ECS_SECURITY_GROUP_ID>],assignPublicIp=DISABLED}" \
  --load-balancers \
    "targetGroupArn=<BLUE_TARGET_GROUP_ARN>,containerName=quiz-app,containerPort=80" \
  --region <AWS_REGION>

```

Create the Green Service

```
aws ecs create-service \  
  --cluster capstone-cluster \  
  --service-name quiz-app-green \  
  --task-definition quiz-app \  
  --desired-count 1 \  
  --launch-type FARGATE \  
  --network-configuration \  
    "awsvpcConfiguration={subnets=[<PRIVATE_SUBNET_A>,<PRIVATE_SUBNET_B>],securityGroups=[<ECS_SECURITY_GROUP_ID>],assignPublicIp=DISABLED}" \  
  --load-balancers \  
    "targetGroupArn=<GREEN_TARGET_GROUP_ARN>,containerName=quiz-app,containerPort=80" \  
  --region <AWS_REGION>
```

DescribeServices					
quiz-app-blue	ACTIVE	1	1	0	
quiz-app-green	ACTIVE	1	1	0	

6.3 Deploy the New Application Version

After the validated Docker image has been pushed to Amazon ECR, the Green ECS service is updated to use the new task definition.

The task definition references the Docker image using the Jenkins build number. This allows the deployment pipeline to deploy the exact image that passed the application validation and security checks.

Update the Green ECS service with the registered task definition.

```
aws ecs update-service \  
  --cluster capstone-cluster \  
  --service quiz-app-green \  
  --task-definition quiz-app \  
  --region <AWS_REGION>
```


Add the deployment stage after the existing Push to ECR stage.

```
stage('Deploy to Green') {  
  steps {  
    sh '''  
      aws ecs update-service \  
        --cluster capstone-cluster \  
        --service quiz-app-green \  
        --task-definition quiz-app \  
        --region ${AWS_REGION}  
    '''  
  }  
}
```

```
[Pipeline] // node  
[Pipeline] End of Pipeline  
Finished: SUCCESS
```

The Green ECS service now starts tasks using the new application image while the Blue environment remains available.

6.4 Validate the Green Environment

Verify that the new ECS task has started successfully and that the Green target group reports a healthy target.

Jenkinsfile

```
stage('Validate Green') {  
  steps {  
    sh '''  
      aws ecs describe-services \  
        --cluster capstone-cluster \  
        --services quiz-app-green \  
        --region ${AWS_REGION}  
      aws elbv2 describe-target-health \  
        --target-group-arn ${GREEN_TARGET_GROUP_ARN} \  
        --region ${AWS_REGION}  
    '''  
  }  
}
```

```
"status": "ACTIVE",
"desiredCount": 1,
"runningCount": 1,
"pendingCount": 0,
"launchType": "FARGATE",
"platformVersion": "LATEST",
"platformFamily": "Linux",
```

```
"TargetHealth": {
  "State": "healthy"
},
```

The Green environment is considered ready when the ECS task is running and the target group reports the task as healthy. The Blue environment continues to serve the existing application version during this validation process.

6.5 Shift Traffic from Blue to Green

Update the Application Load Balancer listener to forward traffic to the Green target group.

Before the traffic shift:

```
ALB → Blue Target Group → Blue ECS Service
```

The traffic shift is automated through Jenkins by updating the Application Load Balancer listener to use the Green target group as its default target.

Jenkinsfile

```
stage('Shift Traffic to Green') {
  steps {
    sh '''
      aws elbv2 modify-listener \
        --listener-arn ${LISTENER_ARN} \
        --default-actions \
          Type=forward,TargetGroupArn=${GREEN_TARGET_GROUP_ARN} \
        --region ${AWS_REGION}
    '''
  }
}
```

Add the following variables to the environment section.

```
LISTENER_ARN = '<LISTENER_ARN>'
GREEN_TARGET_GROUP_ARN = '<GREEN_TARGET_GROUP_ARN>'
```

After the traffic shift:

```
ALB → Green Target Group → Green ECS Service
```

```
[Pipeline] { (Shift Traffic to Green)
[Pipeline] sh
+ aws elbv2 modify-listener --listener-arn arn:aws:elasticloadbalancing:ap-south-1:
```

The Blue environment remains available after the traffic shift, allowing traffic to be returned to the previous application version if a rollback is required.

6.6 Verify Rollback

If an issue is detected after traffic is shifted to Green, restore the Application Load Balancer listener to the Blue target group.

The traffic flow is restored from

```
ALB → Green Target Group → Green ECS Service
```

to:

```
ALB → Blue Target Group → Blue ECS Service
```

Jenkinsfile

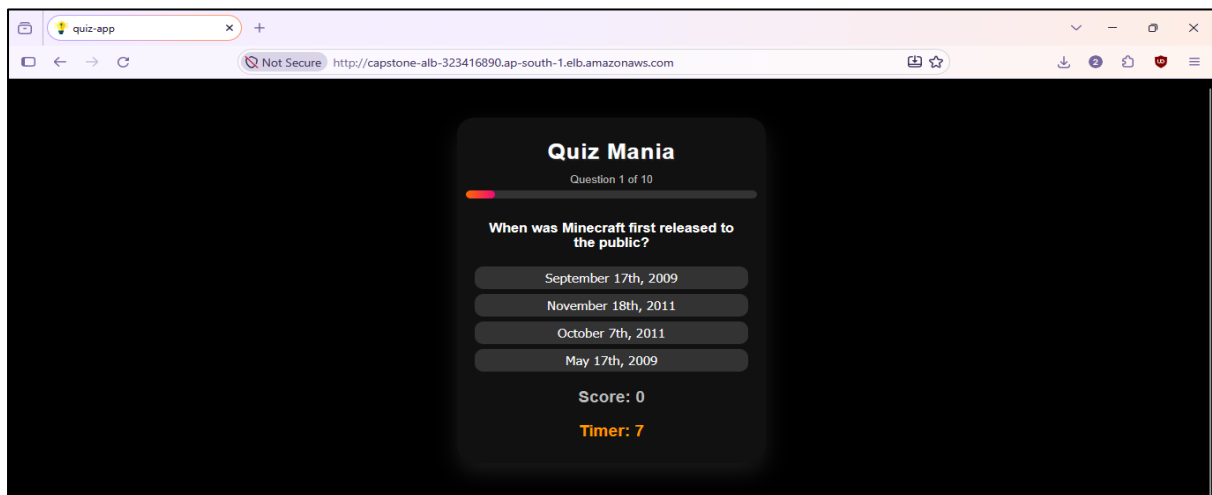
```
stage('Rollback to Blue') {
    steps {
        input message: 'Rollback traffic to Blue environment?'
        sh '''
            aws elbv2 modify-listener \
            --listener-arn ${LISTENER_ARN} \
            --default-actions \
            Type=forward,TargetGroupArn=${BLUE_TARGET_GROUP_ARN} \
            --region ${AWS_REGION}
        '''
    }
}
```

```
Rollback traffic to Blue environment?  
Proceed or Abort  
Approved by Admin  
[Pipeline] sh  
+ aws elbv2 modify-listener --listener-arn arn:aws:elasticloadbalancing:ap-south-1:
```

Open the ALB DNS name in a browser.

```
http://<ALB_DNS_NAME>
```

The Quiz App should now be accessible through the Application Load Balancer.



Step 7: Monitoring and Observability

CloudWatch collects information from the ECS application and Application Load Balancer, while **Grafana** runs on the Managed EC2 instance.

7.1 Configure CloudWatch Logging

The ECS task definition uses the AWS Logs driver to forward application output to the configured CloudWatch log group.

ecs-task-definition.json

```
{  
  "logConfiguration": {  
    "logDriver": "awslogs",  
    "options": {  
      "awslogs-group": "/ecs/capstone",  
      "awslogs-region": "<AWS_REGION>",  
      "awslogs-stream-prefix": "ecs"  
    }  
  }  
}
```

The ECS container logs are collected under the `/ecs/capstone` log group, allowing application output to be reviewed from the CloudWatch console.

Verify the log group.

```
aws logs describe-log-groups \
  --log-group-name-prefix /ecs/capstone \
  --region <AWS_REGION>
```

```
"logGroupName": "/ecs/capstone",
"creationTime": 1791445447323,
"retentionInDays": 7,
"metricFilterCount": 0,
"arn": "arn:aws:logs:ap-south-1:"
```

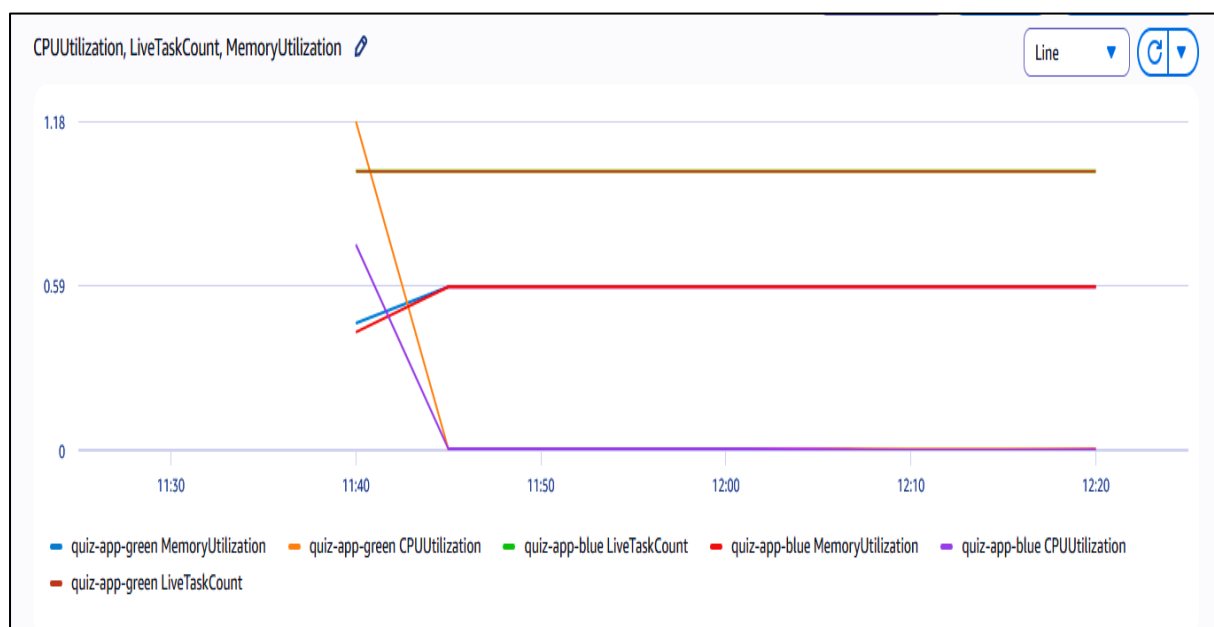
7.2 Configure CloudWatch Metrics

Amazon CloudWatch metrics are used to monitor the health and performance of the ECS application and its supporting AWS resources.

Monitor the following metrics for the **ECS service**, **running tasks**, and **Application Load Balancer**.

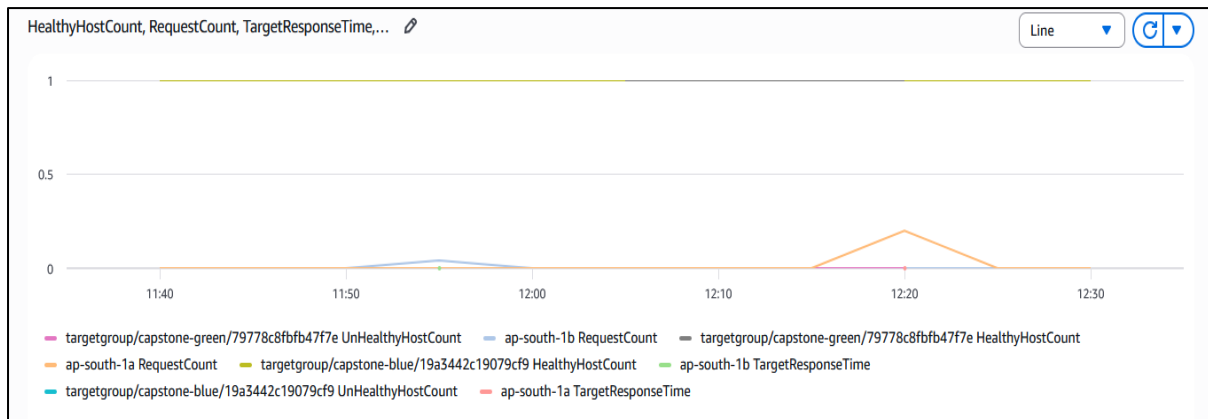
ECS

CPU Utilization, Memory Utilization, Running Task Count



Application Load Balancer

Request Count, Target Response Time, Healthy Host Count, Unhealthy Host Count



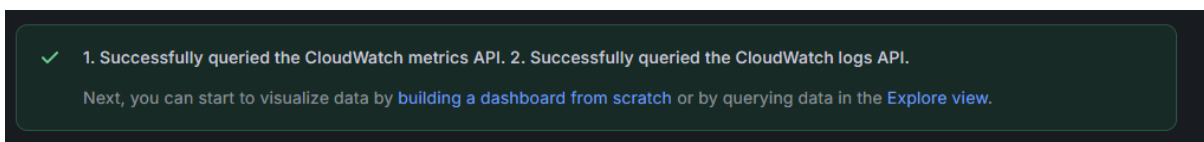
7.3 Configure Grafana

Configure **Amazon CloudWatch** as the **Grafana data source**. Grafana retrieves monitoring data directly from CloudWatch and uses it to create dashboards for the ECS services and Application Load Balancer.

In Grafana:

1. Open **Connections / Data Sources**.
2. Select **Add data source**.
3. Select **Amazon CloudWatch**.
4. Configure the AWS region as `<AWS_REGION>`.
5. Configure authentication using the AWS permissions available to the Managed EC2.
6. Select **Save & Test**.

The Managed EC2 uses an IAM role with CloudWatch read access, allowing Grafana to retrieve CloudWatch metrics without storing AWS access keys on the server.



iam.tf

```
resource "aws_iam_role_policy_attachment" "grafana_cloudwatch" {
  role = aws_iam_role.grafana.name
  policy_arn = "arn:aws:iam::aws:policy/CloudWatchReadOnlyAccess"
}
```

ec2.tf

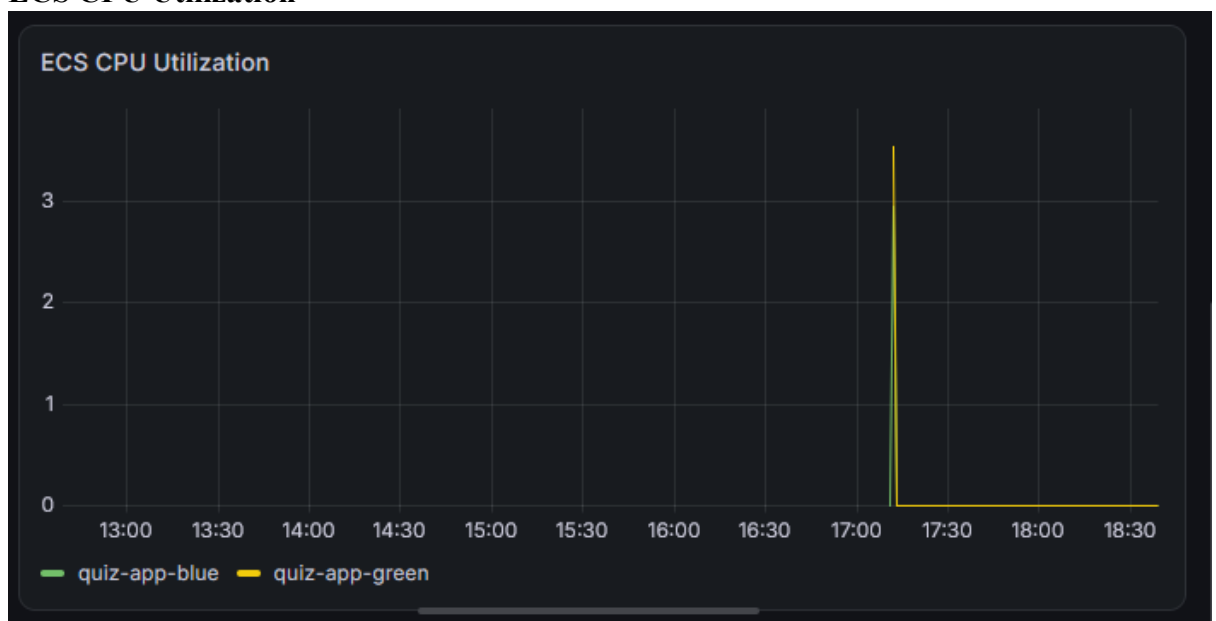
```
resource "aws_instance" "managed" {
  ami          = var.ami_id
  instance_type = var.instance_type
  subnet_id    = aws_subnet.public[0].id
  key_name     = var.key_name
  iam_instance_profile = aws_iam_instance_profile.grafana.name
  vpc_security_group_ids = [
    aws_security_group.managed_ec2.id
  ]
  associate_public_ip_address = true
  tags = {
    Name = "${var.project_name}-managed-ec2"
  }
}
```

7.4 Create Monitoring Dashboard

Create a Grafana dashboard using the configured CloudWatch data source.

Configure dashboard panels for the following.

- **ECS CPU Utilization**



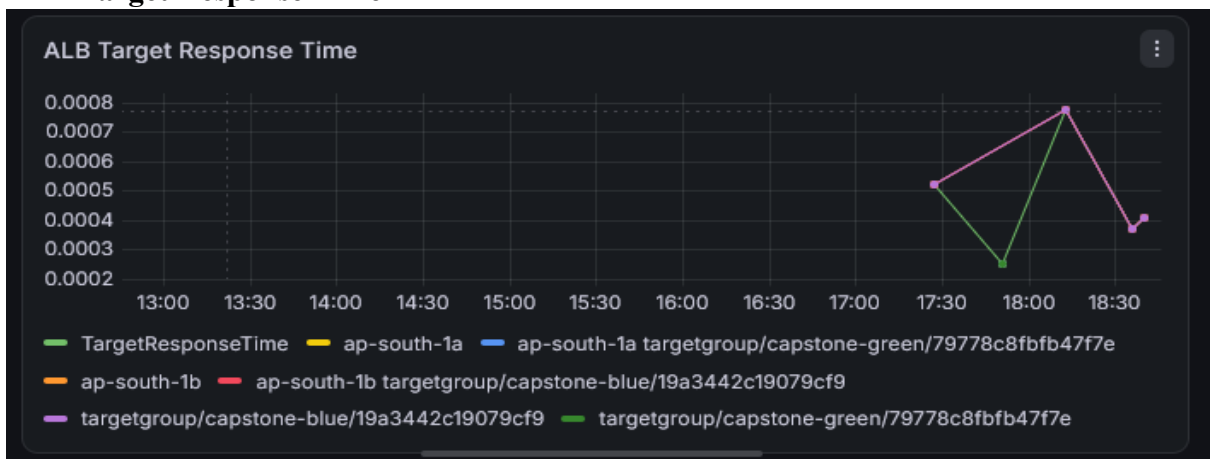
- **ECS Memory Utilization**



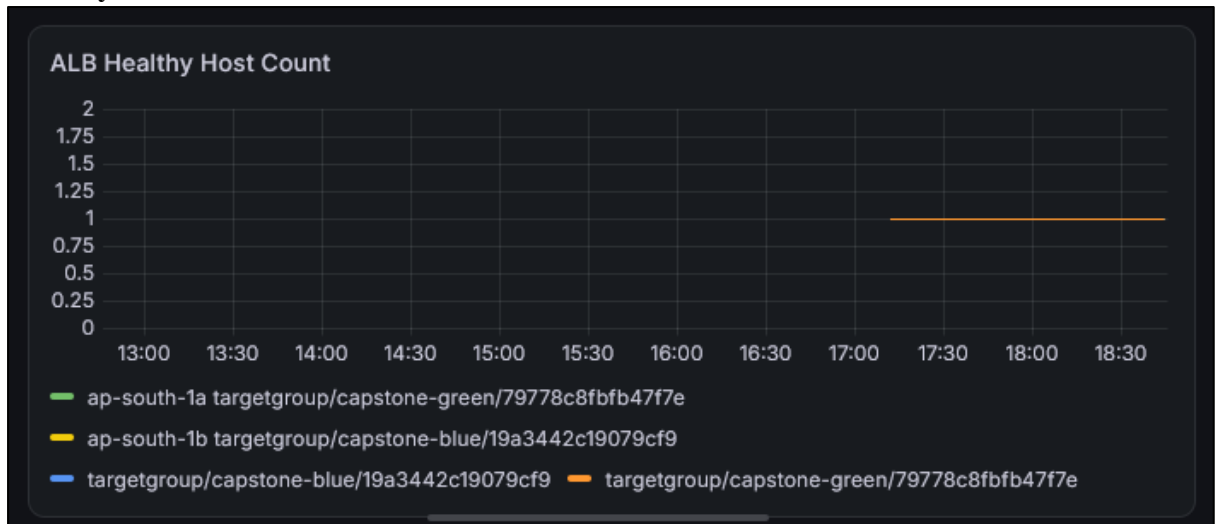
- **ALB Request Count**



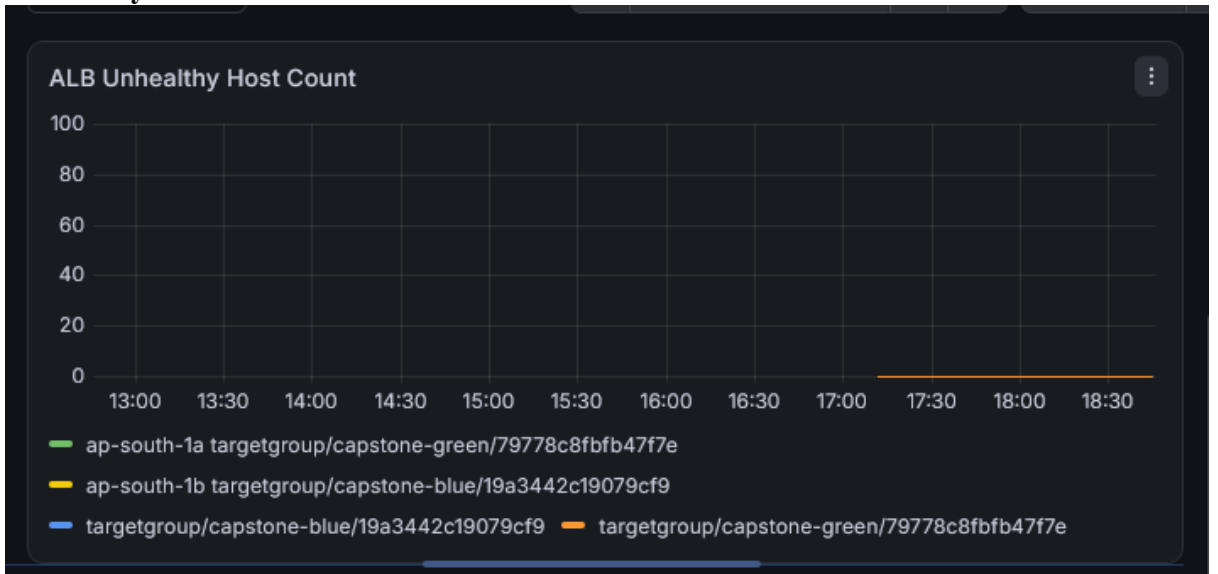
- **ALB Target Response Time**



- **Healthy Host Count**



- **Unhealthy Host Count**



Keep **Amazon CloudWatch** as the data source for the dashboard. The ECS panels use the **AWS/ECS** namespace and the Application Load Balancer panels use **AWS/ApplicationELB** namespace.

7.5 Configure and Test Grafana Alerts

Create an alert rule from the Grafana dashboard.

Choose **Edit** → **Alert** → **Create alert rule**

Two alerts can be configured for the monitoring dashboard.

Alert 1: High ECS CPU Utilization

The alert monitors the **CPUUtilization** metric from the **AWS/ECS** namespace.

Data Source: Amazon Cloudwatch
Namespace: AWS/ECS
Metric: CPUUtilization
Statistic: Average
ClusterName: capstone-cluster
ServiceName: quiz-app-blue

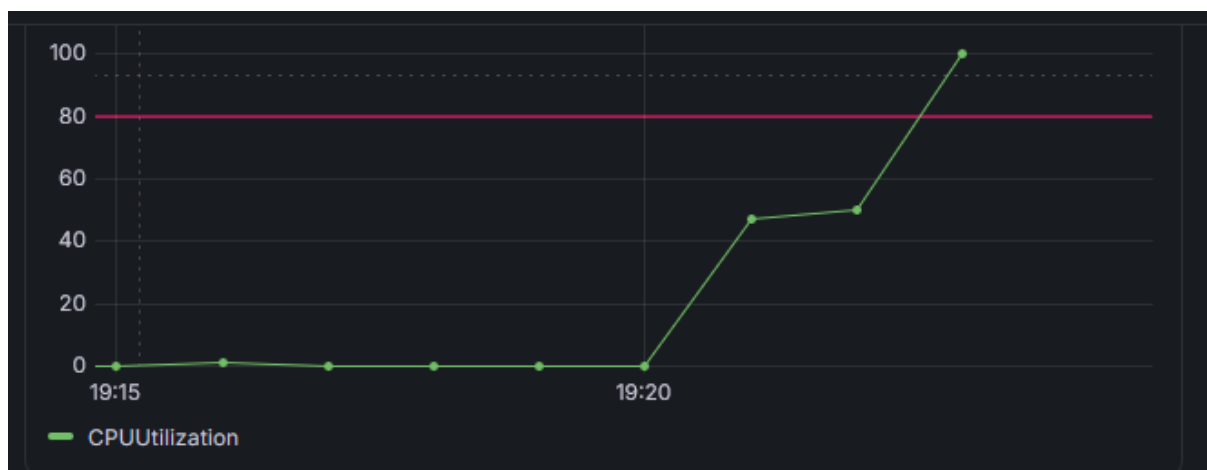
Alert 2: Unhealthy ALB Targets

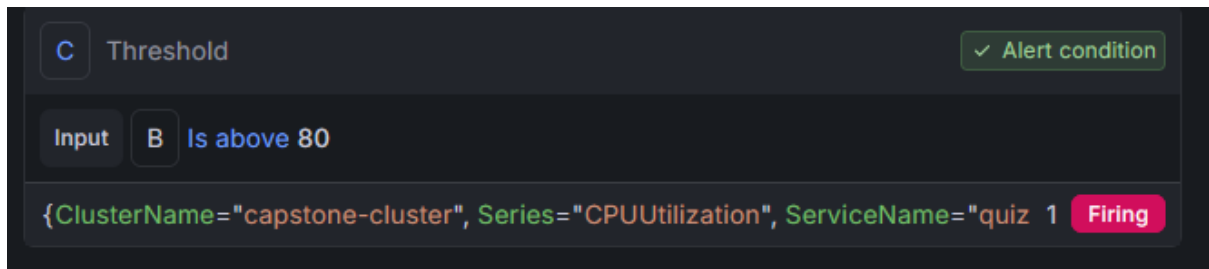
The alert monitors the **UnHealthyHostCount** metric from the **AWS/ApplicationELB** namespace.

Data source: Amazon CloudWatch
Namespace: AWS/ApplicationELB
Metric: UnHealthyHostCount
TargetGroup: blue-target-group
LoadBalancer: capstone-alb
Condition: Greater than 0

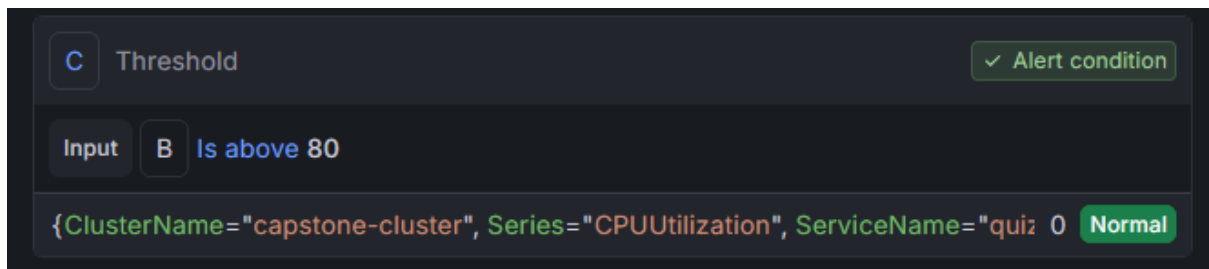
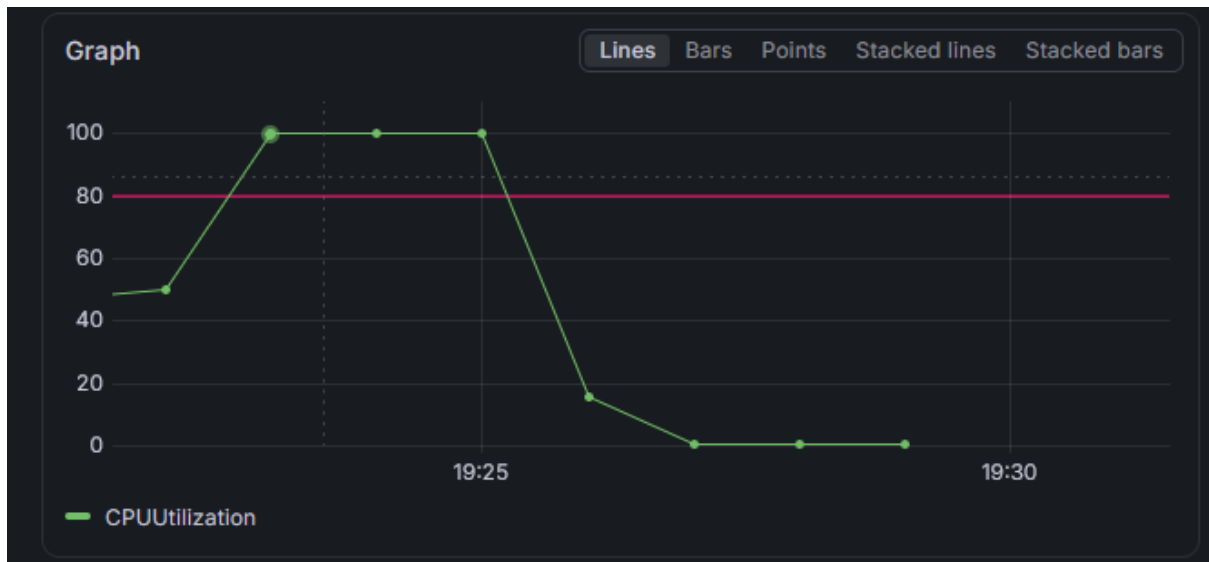
Test the Grafana Alert

For the **High ECS CPU Utilization** alert, a temporary **CPU stress test** is performed on the running ECS task to generate sustained CPU usage

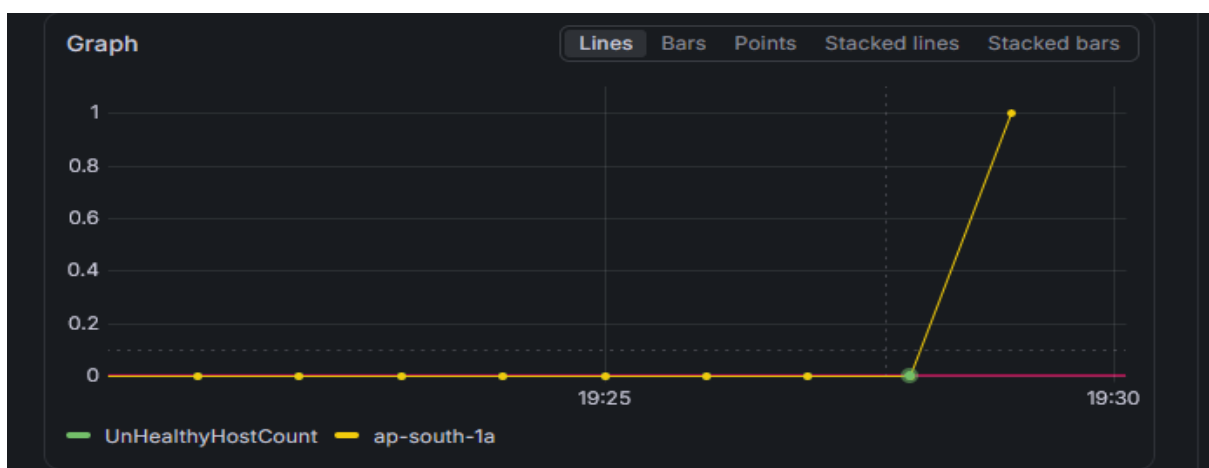


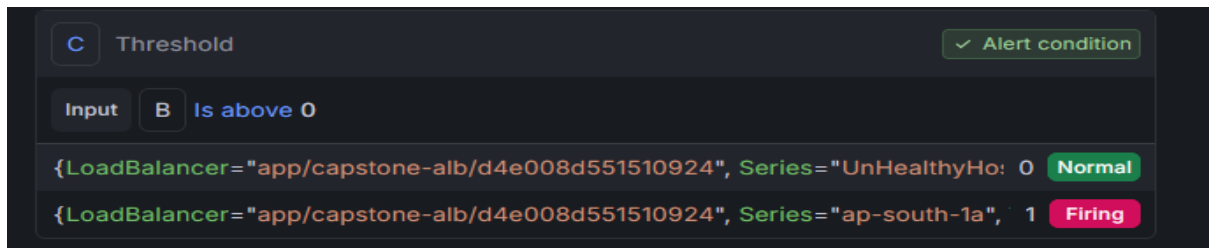


After a few moments, it comes back to normal

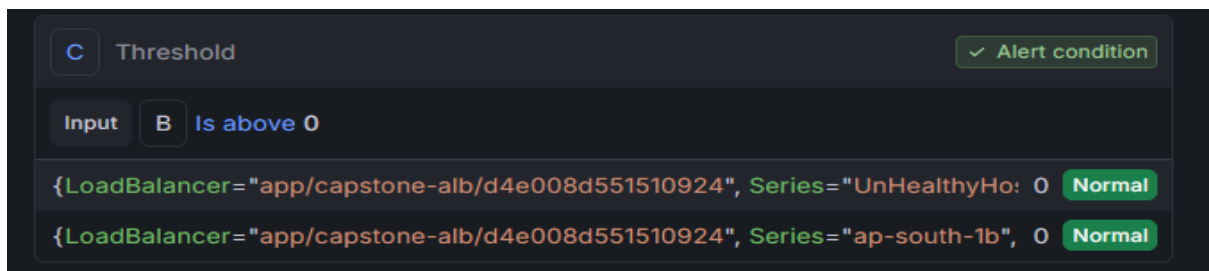
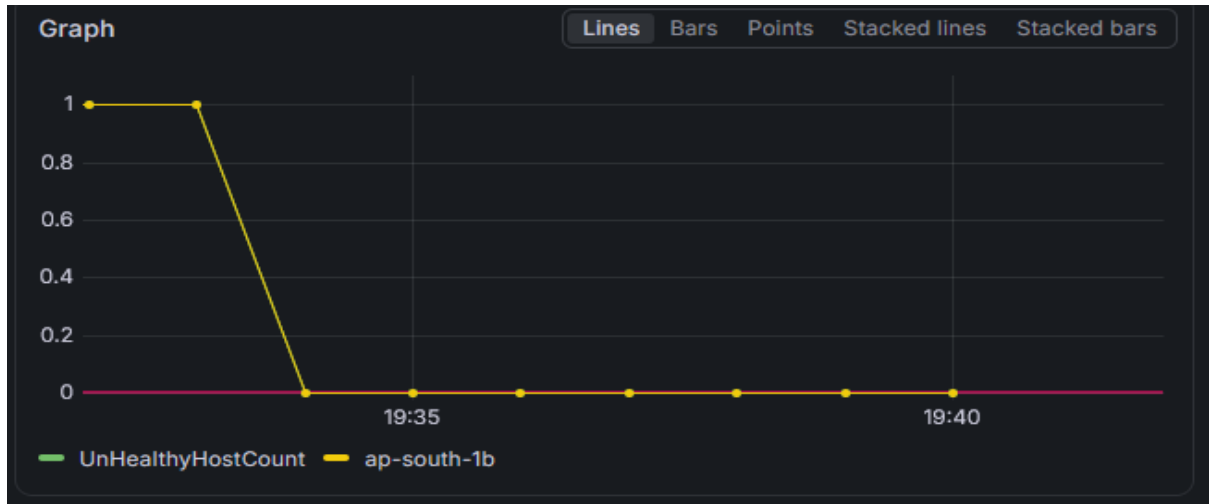


For the **Unhealthy ALB Targets** alert, an ECS target is temporarily made unavailable.





After a few moments, it comes back to normal



Conclusion

This project implements an automated application delivery workflow on AWS using Jenkins, Terraform, Ansible, Docker, Amazon ECR, and Amazon ECS. The React + Vite Quiz App is built, tested, security-scanned, containerized, and deployed through Jenkins, with Blue/Green deployment used to manage application releases.

AWS CloudWatch and Grafana provide monitoring and observability for the deployed application. Grafana dashboards and alerts are used to monitor ECS and ALB metrics, with alert testing performed for high ECS CPU utilization and unhealthy ALB targets.